NPM fundamentals - installing and managing packages

Sure! Here's a **detailed explanation** of **NPM fundamentals** — how to **install and manage packages** in Node.js.

🌐 What is NPM?

NPM stands for **Node Package Manager**. It's the **default package manager for Node.js**, allowing developers to:

- Install third-party libraries (called packages)
- Manage versioning
- Share and reuse code

NPM also hosts the **npm registry**, a massive database of over a million packages.

📦 Installing NPM

- **Automatically installed** with Node.js
- Verify installation:

```
node -v
npm -v
```

📁 `package.json` — The Project Manifest

Before installing packages, initialize a project:

```
npm init
```

or for default settings:

```
npm init -y
```

This creates a `package.json` file containing metadata like:

- Project name, version, author
- Scripts (e.g., `start`, `test`)
- Dependencies and devDependencies

📥 Installing Packages

✅ Local Installation (default)

Installs the package in the `node_modules` folder and adds it to `package.json`.

```
npm install <package-name>
# or shorthand
npm i <package-name>
```

Example:

```
npm install express
```

→ Adds this in `dependencies`.

⚙️ Dev Dependencies

Used only in development (e.g., testing or build tools):

```
npm install nodemon --save-dev
# or shorthand
npm i nodemon -D
```

Saved under `devDependencies`.

📤 Global Installation

Used for CLI tools you run from the terminal anywhere:

```
npm install -g <package-name>
```

Example:

```
npm install -g nodemon
```

Use this when the package is a CLI tool you'll use outside a specific project.

📌 Installing Specific Versions

You can install a particular version of a package:

```
npm install express@4.18.2
```

You can also use semantic versioning like `^` and `~`:

- `^1.2.3`: Accepts minor/patch updates (e.g., `1.x.x`)
- `~1.2.3`: Accepts patch updates only (e.g., `1.2.x`)

🗑️ Uninstalling Packages

```
npm uninstall <package-name>
# or shorthand
npm remove <package-name>
```

🔄 Updating Packages

Update an installed package to the latest version:

```
npm update <package-name>
```

Or all packages:

```
npm update
```

📁 The `node_modules` Folder

This is where all installed packages are stored locally.

> ⚠️ This folder should not be committed to Git — it's usually in .gitignore.

📦 Managing the `package-lock.json`

- Created automatically when packages are installed
- Locks the exact versions to ensure consistent installs
- Don't delete this — it ensures your project behaves the same on all machines

---

🛠️ NPM Scripts

Inside `package.json`, under `"scripts"`, you can define command-line tasks.

Example:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

Run it with:

```
npm start
npm run dev
```

📌 Common NPM Commands

Command	Description

`npm init`	Creates `package.json`
`npm install` / `npm i`	Installs all dependencies
`npm install <pkg>`	Installs a package
`npm uninstall <pkg>`	Removes a package
`npm update`	Updates all packages
`npm list`	Lists installed packages
`npm outdated`	Shows outdated packages
`npm run <script>`	Runs a script from `package.json`
`npm cache clean --force`	Clears the npm cache

✅ Best Practices

- Use `package.json` for reproducibility
- Avoid committing `node_modules`
- Use `npm ci` in CI pipelines (faster, uses `package-lock.json`)
- Use `-save-dev` for development-only tools

Let me know if you'd like a visual breakdown, a cheatsheet, or an interactive project to try out NPM commands!